



MCP
Dev Summit
Seoul

Building and Taming an MCP Server for Multi-Cloud Vibe Computing

Ph.D. Seokho Son

- ETRI Special Fellow
- CNCF Ambassador

Who am I



 seokho-son

 seokho-son

 seokho son (CNCF / K8s Slack)

Ph.D. in Computer Science,

Senior Researcher & Special Fellow,



Ambassador,  **CLOUD NATIVE
COMPUTING FOUNDATION**



Maintainer, CNCF Cloud Native Glossary



Maintainer, Kubernetes Website



Lead, Kubernetes SIG-Docs Subproject



Lead, Kubernetes SIG-Docs Korean I10n Team



Maintainer, M-CMP Platform



Maintainer, Cloud-Barista



Who am I



WooJu Son (손우주) D+96

우주아빠
father of
the universe

"WooJu" means
Universe (Space)

Understanding the title...

Building and Taming an MCP Server for Multi-Cloud Vibe Computing ??

Vibe Computing: A New Paradigm Beyond the Cloud

“Don’t configure the cloud. Feel the vibe.”

Concept introduced by [Ph.D. Seokho Son](#) on August 3, 2025

Overview

Vibe Computing is a newly proposed paradigm that succeeds traditional cloud computing. It extends the idea of computing freedom by enabling users to create and control any type of infrastructure—virtual machines, containers, serverless functions, or physical clusters—through intuitive vibes alone. It envisions a computing experience where users no longer need to understand or manage complex cloud infrastructures. Instead, users operate through intuitive 'vibes', delegating the orchestration of resources and services to an intelligent and automated backend.

While **Vibe Coding** refers to the practice of prompting large language models (LLMs) to generate code via natural language, **Vibe Computing** elevates this further—automating not just coding, but the entire infrastructure lifecycle and execution pipeline based on the user’s goal. It’s a meta-computing strategy built on intent recognition and AI-driven automation. In the long term, this approach could be extended to incorporate even futuristic computing paradigms such as quantum computing or neuromorphic systems, offering users the ability to orchestrate not only today’s infrastructure but also tomorrow’s breakthroughs—through vibes alone.

Evolution of Computing Paradigms

The development of computing paradigms has expanded from centralized to distributed, and now to intent-based orchestration. Vibe Computing emerges as a natural evolution from prior models:

- Grid Computing → Cloud Computing (abstracting infrastructure complexity as a service)
- → **Vibe Computing** (abstracting the entire stack—users no longer manage, they simply intend)

[Github.com/seokho-son/vibe-computing](https://github.com/seokho-son/vibe-computing)

Grid Computing

→

Cloud Computing

(abstracting infra complexity as a service)

→

Vibe Computing

(abstracting the entire stack — users no longer manage, they simply intend)

“Computing paradigm that can be consumed by AI”



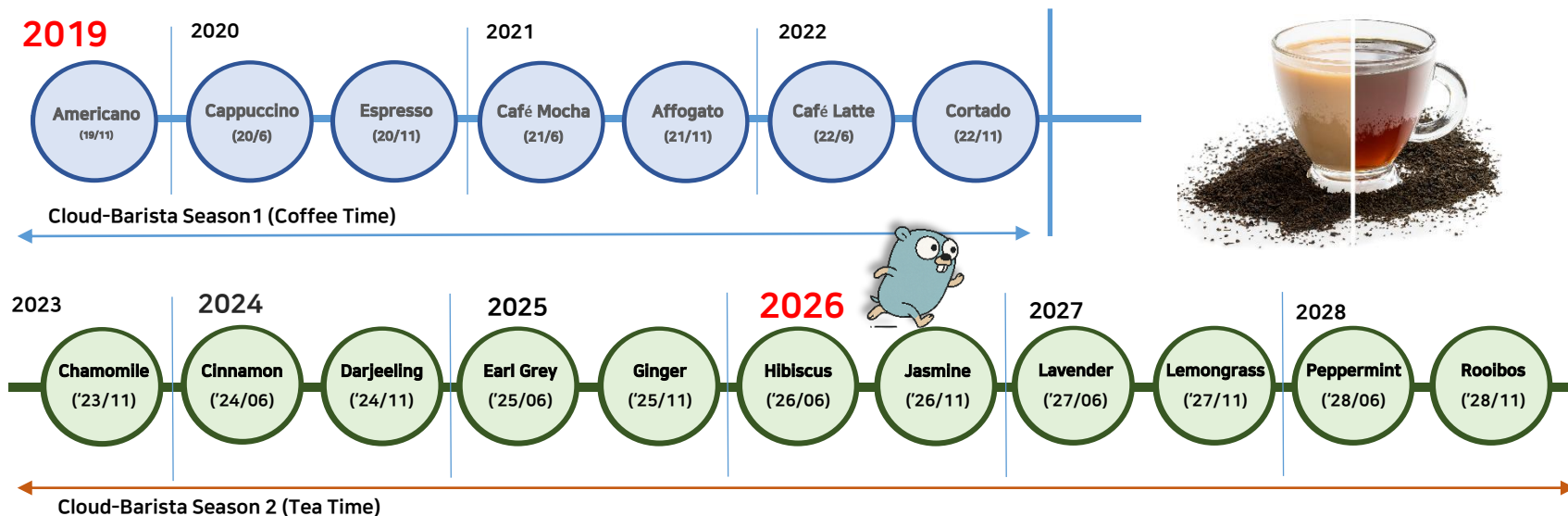
In fact, I'm going to talk about ...

Cloud-Barista
(Multi-Cloud) + **MCP & LLM**
(AI Interface) **Vibe**
Computing

story about adopting MCP to Cloud-Barista

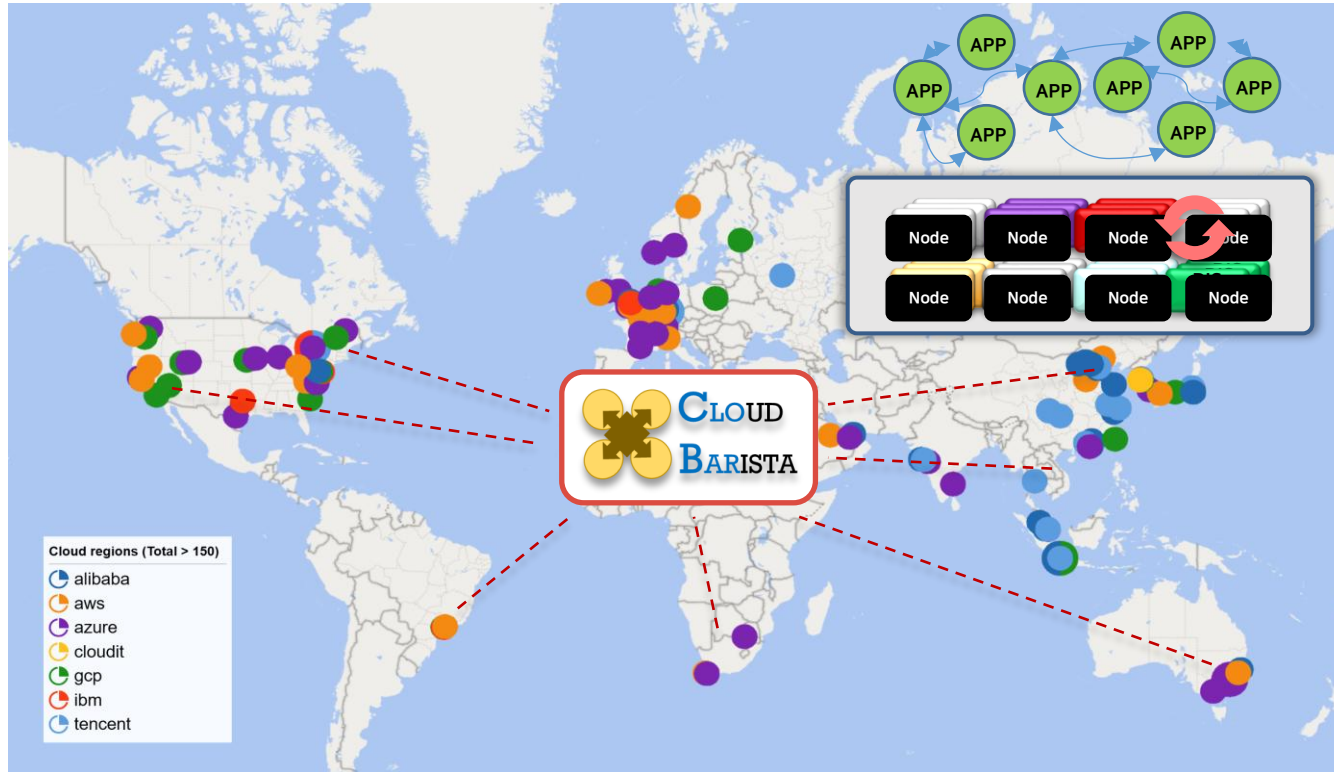
Cloud-Barista is an Open Source (Apache 2.0)

Project and community to provide essential functionalities for building multi-cloud solutions



github.com/cloud-barista/cb-tumblebug

Manages Multi-Cloud Infrastructures !



Multi-Cloud Application

Multi-Cloud Computing Infrastructure



computing resource

computing resource

computing resource

Heterogeneous Clouds abstraction



Extending supported CSPs and Resource types

CONTEXT

Supported Resource Types											
Resource Type	AWS	Azure	GCP	Alibaba	Tencent	IBM	NCP	NHN	KT	Open Stack	
VM (CPU)	YES	YES	YES	YES	YES	YES	YES	Partial	Partial	YES	
VM (GPU)	YES	YES	YES	YES	WIP	TBD	TBD	TBD	TBD	TBD	
Virtual Network	YES	YES	YES	YES	YES	YES	YES	Partial	Partial	YES	
Security Group	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES	
SSH key	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES	
Volume (Disk)	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES	
NLB (L4 LB)	YES	YES	YES	YES	YES	YES	YES	WIP	Partial	YES	
Kubernetes	YES	YES	YES	YES	YES	TBD	TBD	YES	NA	NA	
AWS-site VPN	YES	YES	YES	YES	YES	YES	TBD	TBD	TBD	TBD	

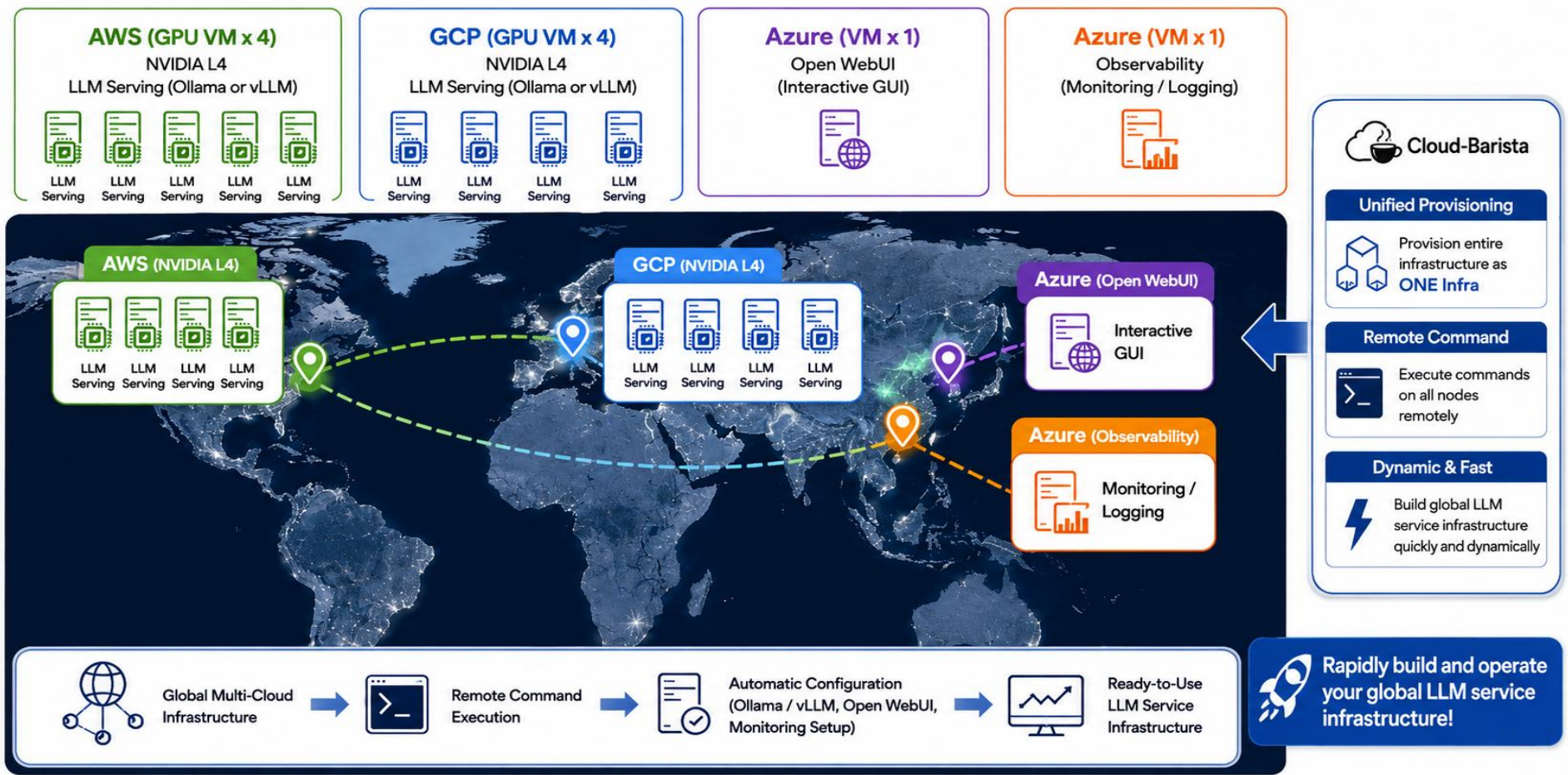
(Note: reference only, and we do not guarantee its functionality. It is regularly updated.)

[Live doc: CB-TB Supported Resource Types](#)

REST API: 200+
(POST:66 GET:75 PUT:14 DEL:42)

Cloud-Barista: Multi-LLMs demonstration

Provision and configure a global LLM service infra dynamically



```
son@SON:~/go/src/github.com/cloud-barista/cb-tumblebug$  
son@SON:~/go/src/github.com/cloud-barista/cb-tumblebug$  
son@SON:~/go/src/github.com/cloud-barista/cb-tumblebug$  
son@SON:~/go/src/github.com/cloud-barista/cb-tumblebug$  
son@SON:~/go/src/github.com/cloud-barista/cb-tumblebug$  
son@SON:~/go/src/github.com/cloud-barista/cb-tumblebug$ m
```



cloud-barista

opens API server.

"make up" "make init"

make everything ready.

MCP
ANNOUNCED
NOV 25, 2024

FAST ADOPTION

CLOUD-BARISTA MCP
EARLY 2025

Cloud-Barista

10 CLOUDS

TOKENS

1,000
GPU VMs

WRONG
API

47%

REST
POCS

MCP
SERVER

LLM

AI
AGENT

COST



**So, what does
the outcome look like?**

To AI:

Using MCP server, create an infrastructure consisting of four L4 GPU nodes on AWS and four on GCP, configured as a GPU farm. Carefully select images with GPU drivers already installed. Also, create one node on Azure for installing Open WebUI.

Once the infrastructure has been created and is in the running state, install Ollama on all nodes in the GPU farm. Select eight LLM models that can run on the nodes and deploy a different LLM model to each node (prefer tool-supported llm model). Then, install Open WebUI on the Azure VM and configure Ollama so that the models can be used immediately. Also, prepare DNS access to the WebUI.

To AI:

By using MCP, create a global infra with 10 vm nodes distributed world widely for a website that introduce MCP dev summit conference and AAlF foundation in LF.

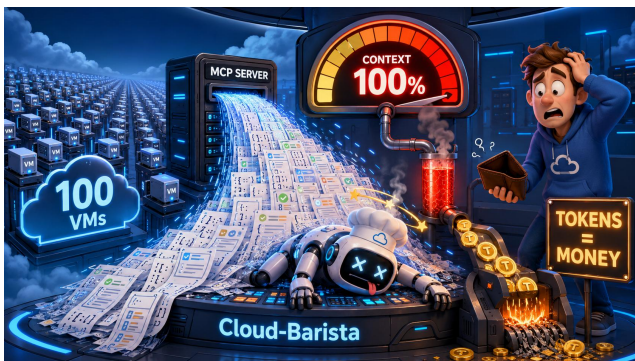
Deploy nginx to all nodes in the Infra and make a webpage for it. You can determine anything. :)

How did we achieve this?

6 problems we faced



FINOPS & GOVERNANCE



TOKEN ECONOMY



SYSTEM HARDENING



MODEL-AGNOSTIC



SESSION CONTINUITY



ZERO-TRUST SECURITY



A ceiling the agent cannot raise

\$326.60 per hour

one machine. one wrong pick.

the most expensive spec in our own catalogue —
and nothing in the first server would have refused it

per creation

\$10 / h

what this one plan may cost

per day

\$100 / h

everything created in 24 hours

running now

\$50 / h

everything alive, right now

make mcp-budget-set PER_CREATION=10 PER_DAY=100 CONCURRENT=50 (off until an operator sets them)

a spec with no published price is allowed, not refused — a guardrail that breaks your own use case is worse than none

Over any of the three, the call is refused and a request id comes back. so who can approve it? →

The agent decides what to build. It does not decide what that may cost.

The agent can ask. It can never grant by self.

Previously, approved by: the model

the first version let the caller lift its own limit

the ceiling lives in server config — the model can read it, argue with it, and never move it

1 the agent asks

create this Infra

2 MCP refuses

over the limit,
here is a request id

3 a person approves

through a channel the
model cannot reach

4 the agent retries

with that id, once,
before it expires

and the refusal hands over the plan, priced

3 nodes · \$0.1709 / h · 0 unpriced

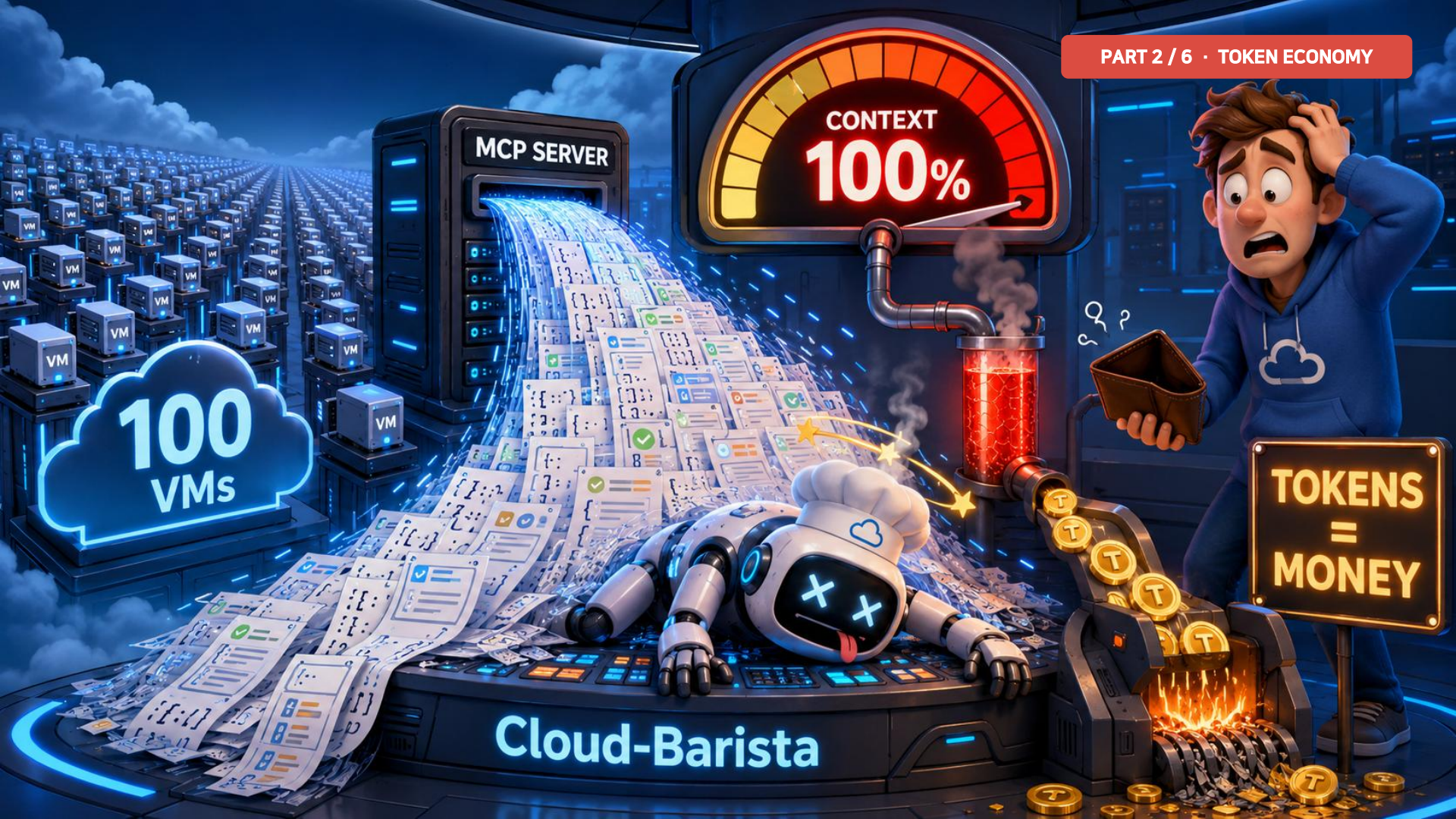
so the person agrees to something specific, not to a category

and the same rule for destruction

terminate_infra(confirm=True)

5 destructive tools refuse without it

An irreversible action on a model's own judgement is a defect in the tool.



MCP SERVER

CONTEXT
100%

100
VMs

Cloud-Barista

TOKENS
=
MONEY

Send it once, and send a summary

119,500 tokens

to read one Infra — and 94% of it was
command history nobody asked for

the same JSON, sent twice

the API returns 237,019 B



content



structuredContent

identical · 478 KB on the wire

1 return it once

content only — 66 tools, one decorator line.

2 and return a summary

summary by default — history becomes a count.

get_infra, one node

477,840 B → 3,002 B

get_connections

223,366 B → 20,110 B

Return a summary by default. Let the model ask for the rest.

Fewer tools, smaller answers, and a dial

26,000 tokens

before the model does any work at all —
the tool list is paid at the start of every session

1 fewer tools

88 → 66

Twelve near-identical tools became four that take a kind and an action.

2 smaller answers

1.17 MB → 43,812 B

Filter in the server: return the matching lines, not the whole log.

3 a dial the caller turns

detail = minimal | summary | full

The model asks for the depth it needs, and pays for that much.

the tool list, every session

103,910 B → 65,300 B

one turn, end to end

145,000 tok → 21,000 tok

Tokens are the budget nobody puts on the invoice.



Fail before you spend, not after

1 a mandatory review gate

`review_infra_dynamic_request()`

Availability, spec-image fit
and cost — before anything
exists.

Creating without it is refused.

2 errors that carry the next step

`_ERROR_NEXT_STEPS`

`ZONE_NOT_DELEGATED` →
“`get_dns_hosted_zones()`
lists usable_domains”

The agent recovers alone.

3 a memory of what failed

`get_provisioning_risk(...)`

A combination that failed
before is reported before the
next attempt, with an
alternative.

One review call costs a second.

One failed provision costs a VNet, a security group, a key pair — and the turn that discovers it.

A trial-and-error loop is the most expensive way to learn what the API already knew.

MACHINE TYPES		IMAGES		GPU SPECS		REGIONS		PRICE COVERAGE	
87,956 across 10 providers		203,049 AWS is 98% of it		2,189 2.5% of all types		193 connections verified		95% KT 14%, NCP 45%	
PROVIDER	MACHINE TYPES	IMAGES	OF WHICH BASIC	PRICED	MEDIAN \$/H	PEAK \$/H	GPU SPECS	ACCELERATORS OFFERED	WHAT STANDS OUT
AWS 33 regions	21,062	200,406	8,234	98%	\$1.97	\$327	693	B300 · B200 · H200 · H100 · L40S · L4	Widest everything. 200k images is 98% of the whole image catalogue — AWS publishes one AMI per region per build, so the count is region-multiplied rather than richer.
Azure 48 regions	44,693	851	23	95%	\$1.84	\$274	604	H200 · H100 · MI300X · A100	Most specs of any provider and the deepest memory line, but only 851 images: Azure images are region-agnostic URNs, so one entry serves 48 regions.
Google Cloud 43 regions	11,760	249	12	92%	\$1.72	\$135	343	GB300 · GB200 · B200 · H200 · TPU v7x	The only catalogue with TPUs alongside GPUs, and the newest NVIDIA silicon (GB300). Images are region-agnostic too.
Alibaba Cloud 31 regions	4,835	400	8	93%	\$0.55	\$133	193	L20 · A10 · T4 · V100 (+vGPU slices)	Nineteen accelerator models — the most fragmented GPU line, including fractional vGPU slices. Spec listings are filtered to one zone, so counts run low.
Tencent Cloud 17 regions	2,914	148	13	100%	\$0.67	\$38	130	model not reported	Prices are complete (100%) but accelerator models come back unnamed, so GPU selection cannot be driven from the catalogue.
IBM Cloud 13 regions	2,002	761	56	97%	\$1.99	\$118	110	H200 · H100 · Gaudi3 · L40S	Names encode the role (ux/vx memory, gx GPU). The only catalogue carrying Intel Gaudi3.
NAVER Cloud 3 regions	393	67	6	45%	\$0.68	\$6	105	A100 · T4	GPU-dense for its size — 105 of 393 specs — but only 45% carry a price.
NHN Cloud 3 regions	71	129	6	92%	\$0.38	\$35	9	A100 · V100 · T4	Smallest spec catalogue at 71, yet still spans A100 to T4. Every image is flagged Kubernetes-capable.
KT Cloud 1 regions	220	31	0	14%	\$0.58	\$3	2	V100	Prices are the sparsest gap in the catalogue: 31 of 220 specs, 14%. Specs are named by shape (64x256).

What an agent picks from — and it only helps if it is still true

87,956 machine types · 203,049 images · 2,189 GPU specs · 193 regions · 95% priced
 re-fetched and re-validated continuously · spec ↔ image pairs reconciled per provider
 a stale entry costs the agent a whole provisioning attempt



PART 4 / 6 · MODEL-AGNOSTIC



The same request, driven by five different models — every one of them built it

each Infra is named after the model that created it · CB-MapUI, live



SESSION
LOST

MCP SERVER

SESSION LEDGER

AI
AGENT

VM

PART 5 / 6 · SESSION CONTINUITY

RETRY

CREATE
VM

TIMEOUT

Cloud-Barista

Cloud-Barista

COPY

DUPLICATE VMs

VM

VM

VM

VM

VM

VM

State that outlives the session

1 journaling that is not optional

@tool(mutating=True) wraps 25 tools and writes to etcd. The previous version asked the model to remember.

2 idempotency_key on every mutating call

A retry replays the first result instead of provisioning twice. Failures are deliberately not cached.

survives a pod restart

no → yes

3 resume_session(session_id)

created_resources · in_flight · next step.
About a kilobyte. Re-orient, not replay.

proved itself by accident

Three VMs failed at once.
The retry reused the same key. The failure was not replayed. The success was not duplicated.

2 replicas, 8 fresh sessions

8 / 8 consistent, 0 lost

Do not ask the model to remember. Record it for them.



KNOCK!
KNOCK!

MCP
SERVER

AI
AGENT

WORKS
(FOR NOW)

Cloud-Barista

PART 6 / 6 · ZERO-TRUST SECURITY

agentgateway

Agentic AI
Foundation

Three ways a secret gets out

-----BEGIN RSA PRIVATE KEY-----

our first version returned this to the model —
and therefore into every transcript and log

1

in the transcript

whatever a tool hands back

WHAT LEAKED

a tool returned the private key itself,
so it landed in every log and replay

WHAT WE CHANGED

Redaction is the default —
every tool, any depth.
One tool opts out, and says so.

2

on the wire

the credential in transit

WHAT LEAKED

CSP keys were passed in plain text
between client and server

WHAT WE CHANGED

AES-256-CBC per value, under a
fresh key, RSA-OAEP wrapped.
Standard primitives, not ours.

3

at the edge

who is allowed to call at all

WHAT LEAKED

the server could not tell an operator
from a stranger who reached the port

WHAT WE CHANGED

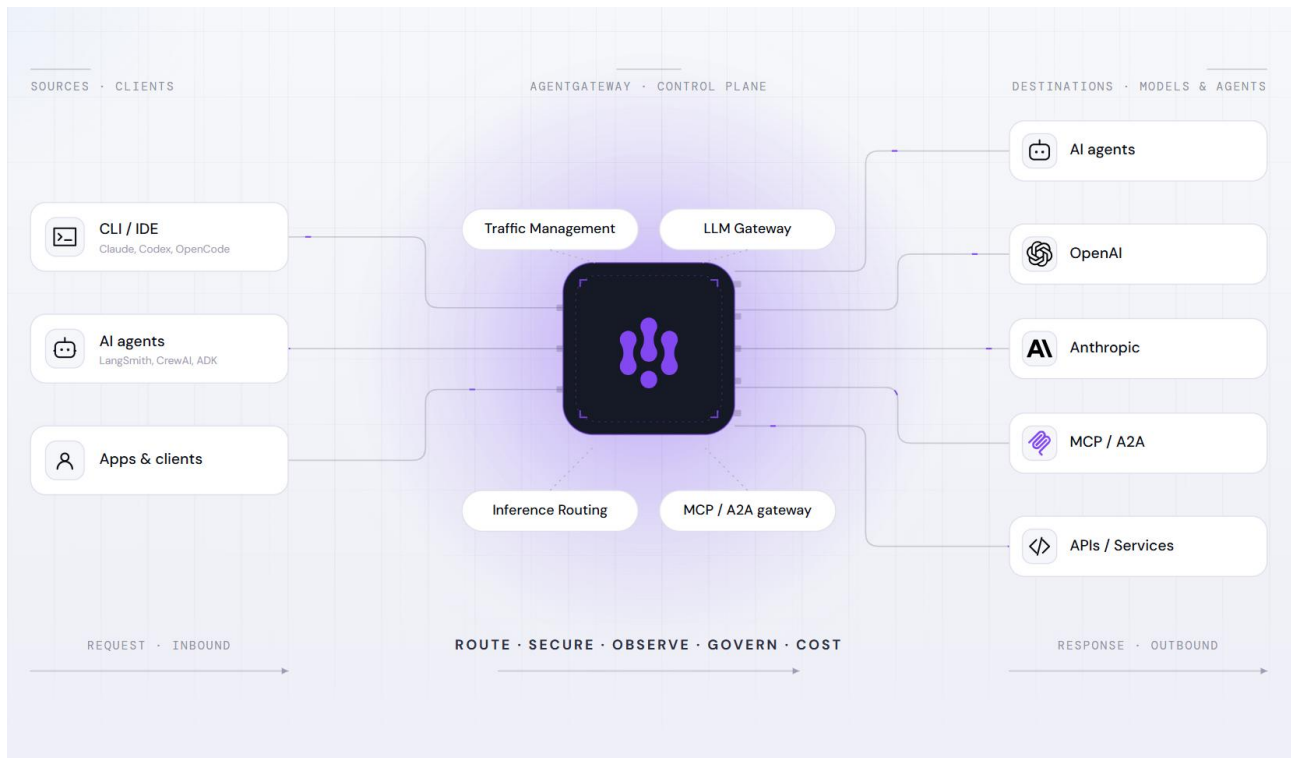
AgentGateway verifies the JWT
and owns the session.
We adopted it, we did not write it.

38-tool sweep after the change: 0 private keys, 0 duplicated payloads, 0 unflagged errors.

Two of these we wrote. The third we did not — and that was the right call.

Open source & Open standard

PART 6 / 6 · ZERO-TRUST SECURITY



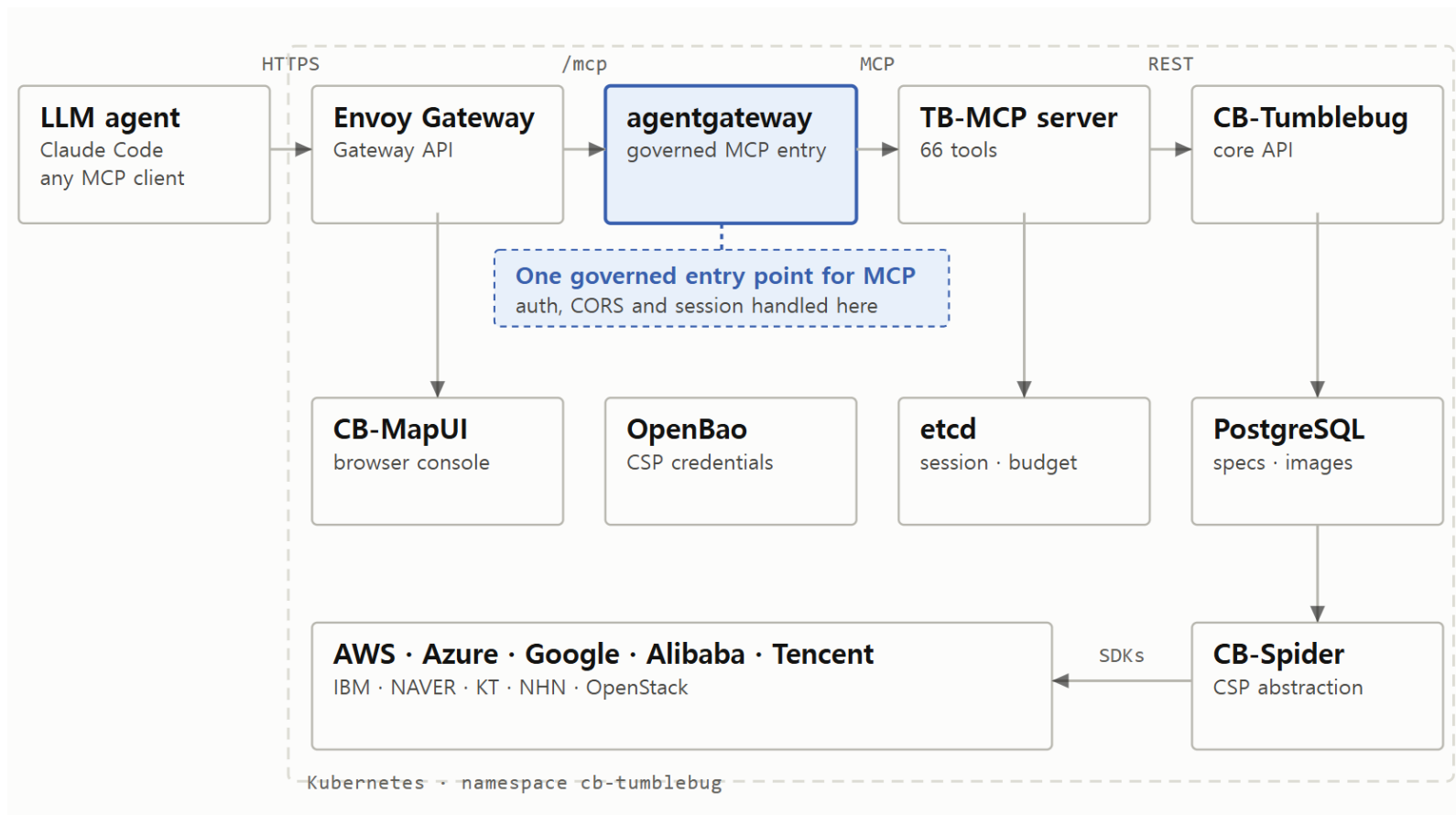
Agentic AI
Foundation



agentgateway

<https://agentgateway.dev>

Cloud-Barista applied agentgateway

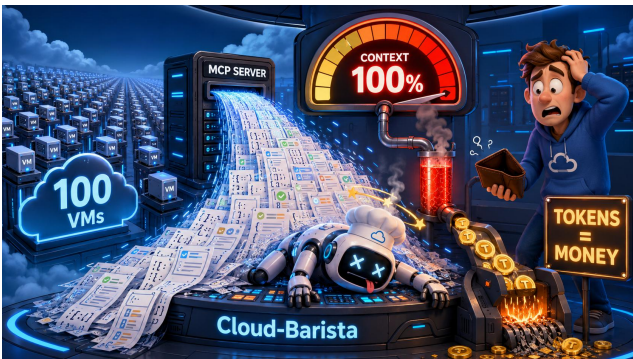


6 problems we resolved



FINOPS & GOVERNANCE

Cap it, then ask a person



TOKEN ECONOMY

A summary by default



SYSTEM HARDENING

System should support (review)



MODEL-AGNOSTIC

Testing loop for all models



SESSION CONTINUITY

Record it for the model



ZERO-TRUST SECURITY

Hand the edge to a standard

If any of this was interesting

CB-Tumblebug (Multi-Cloud Infra Management) 🙋

go report **A+** go **76.4%** go.mod **v1.23.0** repo size **35.1 MiB** **GO** reference Visual Codebase API Doc Swagger

license **Apache-2.0** release **v0.10.0** release(dev) **v0.10.9** build **passing** Slack **SIG-TB**

all contributors **49**

github.com/cloud-barista/cb-tumblebug

★ Open Source Together ★

Your Feedback and Contribution
Keep This Open Source Alive!

CLOUD NATIVE
COMPUTING FOUNDATION

About Projects Training Community Blog & News [Join](#) 🔍

SANDBOX PROJECTS

The CNCF Sandbox is the entry point for early stage projects.

Our Sandbox Projects



★ **planning to apply**

 Aeraki Mesh Service Mesh	 Akri Automation & Configuration	 ANTREA Cloud Native Network	 ARMADA Scheduling & Orchestration	 Athenz Key Management
 Atlantis Automation & Configuration	 bank-vaults Security & Compliance	 BFE Service Proxy	 bpfman Container Runtime	 bpfman Security & Compliance
 Cadence Automation & Configuration	 capsule Scheduling & Orchestration	 Carina Cloud Native Storage	 Cartography Security & Compliance	 CARVEL Application Definition & Image Build



MCP
Dev Summit
Seoul

Thank you!



seokho-son